

基本内容

1. 函数渐近边界

(1) 概念

设 f 和 g 是定义域为自然数集 N 上的函数

(1) $f(n) = O(g(n))$

若存在正数 c 和 n_0 使得对一切 $n \geq n_0$ 有 $0 \leq f(n) \leq cg(n)$

(2) $f(n) = \Omega(g(n))$

若存在正数 c 和 n_0 使得对一切 $n \geq n_0$ 有 $0 \leq cg(n) \leq f(n)$

(3) $f(n) = o(g(n))$

对所有正数 $c > 0$ 存在 n_0 使得对一切 $n \geq n_0$ 有 $0 \leq f(n) < cg(n)$

(4) $f(n) = \omega(g(n))$

对所有正数 $c > 0$ 存在 n_0 使得对一切 $n \geq n_0$ 有 $0 \leq cg(n) < f(n)$

(5) $f(n) = \Theta(g(n)) \Leftrightarrow f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$

(6) $O(1)$ 表示常数函数

(2) 性质

定理 1.1 设 f 和 g 是定义域为自然数集 N 上的函数。

(1) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ 存在，并且等于某个常数 $c > 0$ ，那么

$$f(n) = \Theta(g(n)).$$

(2) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$ ，那么 $\left| \frac{f(n)}{g(n)} \right| < c, n \geq n_0, \forall c > 0.$

$$f(n) = o(g(n)).$$

(3) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = +\infty$ ，那么

$$f(n) = \omega(g(n)).$$

定理 1.2 设 f, g, h 是定义域为自然数集合的函数，

(1) 如果 $f = O(g)$ 且 $g = O(h)$ ，那么 $f = O(h)$ 。

(2) 如果 $f = \Omega(g)$ 且 $g = \Omega(h)$ ，那么 $f = \Omega(h)$ 。

(3) 如果 $f = \Theta(g)$ 和 $g = \Theta(h)$ ，那么 $f = \Theta(h)$ 。

定理 1.3 假设 f 和 g 是定义域为自然数集合的函数，若对某个其它的函数 h ，我们有 $f = O(h)$ 和 $g = O(h)$ ，那么

$$f + g = O(h).$$

推论 假设 f 和 g 是定义域为自然数集合的函数，且满足 $g = O(f)$ ，那么 $f + g = \Theta(f)$ 。

2. 基本函数类

$$2^{2^n}, n!, n2^n, (3/2)^n, (\log n)^{\log n} = \Theta(n^{\log \log n}),$$

$$n^3, \log(n!) = \Theta(n \log n), n = 2^{\log n},$$

$$\log^2 n, \log n, \sqrt{\log n}, \log \log n,$$

$$n^{1/\log n} = \Theta(1)$$

$\log(n^{1/\log n}) = 1 \Rightarrow n^{1/\log n} = \Theta(1)$

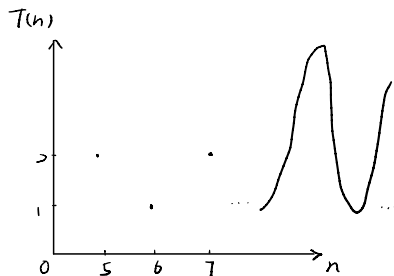
例2 PrimalityTest(n)

输入: n, n 为大于 2 的奇整数

输出: true 或者 false

1. $s \leftarrow \sqrt{n}$
2. for $j \leftarrow 2$ to s
3. if j 整除 n
4. then return false
5. return true

假设计算 $\sqrt{n}$ 可以在 $O(1)$ 时间完成，可以写 $O(\sqrt{n})$ ，质数: $O(\sqrt{n})$ ，不能写 $\Theta(\sqrt{n})$ ← 合数: $O(1)$



3. 数学基础

(1) 对数函数

$$\log_b n = O(n^\alpha), \forall \alpha > 0$$

$$a^{\log_b n} = n^{\log_b a}, a^{\log_b a} = b^{\log_b a \log_b a} = b^{\log_b a} = n^{\log_b a}$$

$$\log_k n = \Theta(\log_l n)$$

(2) 阶乘

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n (1 + \Theta(1/n))$$

$$n! = o(n^n) = \Omega(\alpha^n), \forall \alpha > 1$$

$$\log(n!) = \Theta(n \log n)$$

$$\log n! = \sum_{k=1}^n \log k$$

$$\textcircled{1} \sum_{k=1}^n \log k \geq \int_1^n \log x dx = x \log x \Big|_1^n - \int_1^n x d \log x$$

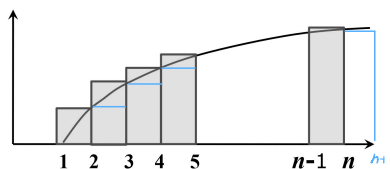
$$= n \log n - \int_1^n \frac{1}{x} dx$$

$$= n \log n - \frac{1}{\log 2} (n-1)$$

$$\Rightarrow \sum_{k=1}^n \log k = \Omega(n \log n)$$

$$\textcircled{2} \sum_{k=1}^n \log k \leq \int_2^{n+1} \log x dx = (n+1) \log(n+1) - \frac{1}{\log 2} (n+3)$$

$$\Rightarrow \sum_{k=1}^n \log k = O(n \log n)$$



(3) 取整函数

$\lfloor x \rfloor$: 表示小于等于 x 的最大的整数

$\lceil x \rceil$: 表示大于等于 x 的最小的整数

取整函数具有下述性质:

(1) $x-1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x+1$

(2) $\lfloor x+n \rfloor = \lfloor x \rfloor + n, \lceil x+n \rceil = \lceil x \rceil + n$, 其中 n 为整数

(3) $\left\lfloor \frac{n}{2} \right\rfloor + \left\lfloor \frac{n}{2} \right\rfloor = n, n$ 为整数

(4) $\left\lfloor \frac{\left\lfloor \frac{n}{a} \right\rfloor}{b} \right\rfloor = \left\lfloor \frac{n}{ab} \right\rfloor, \left\lfloor \frac{\left\lceil \frac{n}{a} \right\rceil}{b} \right\rfloor = \left\lfloor \frac{n}{ab} \right\rfloor$

(4) 求和公式

基本求和公式

$$\sum_{k=1}^n a_k = \frac{n(a_1 + a_n)}{2}, \{a_k\} \text{ 为等差数列}$$

$$\sum_{k=0}^n aq^k = \frac{a(1-q^{n+1})}{1-q}, \sum_{k=0}^n x^k = \frac{1-x^{n+1}}{1-x},$$

$$\sum_{k=0}^{\infty} aq^k = \frac{a}{1-q} \quad (q < 1)$$

$$\sum_{k=1}^n \frac{1}{k} = \ln n + O(1)$$

放大法:

1. $\sum_{k=1}^n a_k \leq na_{\max}$

2. 假设存在常数 $r < 1$ ，使得对一切 $k \geq 0$ 成立 $\frac{a_{k+1}}{a_k} \leq r$ ，则

$$\sum_{k=0}^{\infty} a_k \leq \sum_{k=0}^{\infty} a_0 r^k = a_0 \sum_{k=0}^{\infty} r^k = \frac{a_0}{1-r}$$

$$\sum_{k=1}^{n-1} \frac{1}{k(k+1)} = \sum_{k=1}^{n-1} \left(\frac{1}{k} - \frac{1}{k+1} \right) = \left(1 - \frac{1}{2} + \frac{1}{2} - \frac{1}{3} + \dots + \frac{1}{n-1} - \frac{1}{n} \right) = 1 - \frac{1}{n}$$

$$\sum_{t=1}^k t2^{t-1} = 1 \cdot 2^0 + 2 \cdot 2^1 + \dots + k2^{k-1}$$

$$\Rightarrow \sum_{t=1}^k t2^{t-1} = 1 \cdot 2^1 + \dots + (k-1)2^{k-1} + k2^k$$

$$\Rightarrow \sum_{t=1}^k t2^{t-1} = k2^k - (2^0 + 2^1 + \dots + 2^{k-1})$$

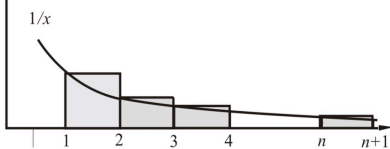
$$= k2^k - \frac{1-2^k}{1-2}$$

$$= k2^k - 2^k + 1$$

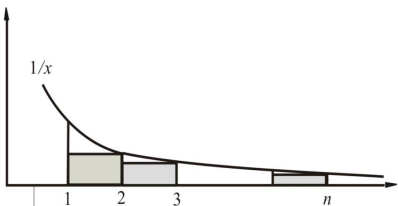
$$= (k-1)2^k + 1$$

估计 $\sum_{k=1}^n \frac{1}{k}$ 的渐近的界.

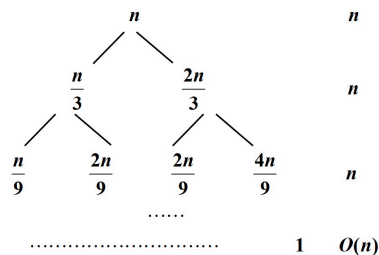
$$\sum_{k=1}^n \frac{1}{k} \geq \int_1^{n+1} \frac{dx}{x} = \ln(n+1)$$



$$\begin{aligned} \sum_{k=1}^n \frac{1}{k} &= \frac{1}{1} + \sum_{k=2}^n \frac{1}{k} \\ &\leq 1 + \int_1^n \frac{dx}{x} \\ &= \ln n + 1 \end{aligned}$$



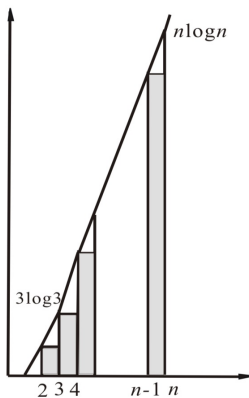
$$T(n) = T(n/3) + T(2n/3) + n$$



$$\begin{aligned} \text{层数 } k: \quad n(2/3)^k = 1 &\Rightarrow (3/2)^k = n \Rightarrow k = O(\log_{3/2} n) \\ T(n) &= O(n \log n) \end{aligned}$$

补充:

$$\begin{aligned} \int_2^n x \log x dx &= \int_2^n \frac{x}{\ln 2} \ln x dx \\ &= \frac{1}{\ln 2} \left[\frac{x^2}{2} \ln x - \frac{x^2}{4} \right]_2^n \\ &= \frac{1}{\ln 2} \left(\frac{n^2}{2} \ln n - \frac{n^2}{4} \right) - \frac{1}{\ln 2} \left(\frac{4}{2} \ln 2 - \frac{4}{4} \right) \\ &= \sum_{i=1}^{n-1} i \log i \\ &= \frac{n^2}{2} \log n - \frac{n^2}{4 \ln 2} + O(1) \end{aligned}$$



递推方程中 $\lfloor x \rfloor$ 和 $\lceil x \rceil$ 的处理

先猜想解, 然后用数学归纳法证明

例16 估计以下递推关系的阶

$$\begin{aligned} T(n) &= 2T(\lfloor \frac{n}{2} \rfloor) + n \\ T(1) &= 1 \end{aligned}$$

$$\text{根据 } T(n) = 2T(\frac{n}{2}) + n$$

$$T(n) = O(n \log n)$$

猜想原递推方程的解的阶是 $O(n \log n)$

证明: $T(n) \leq cn \log n$, 用数学归纳法

证明

归纳基础

对于 $T(1)=1$, 显然没有 $T(1) \leq c \cdot 1 \log 1$.

考虑 $T(2)$

$$T(2) = 2T(\lfloor \frac{2}{2} \rfloor) + 2 = 4 \leq 2 \times 2 \log 2, \quad c=2$$

归纳步骤

假设对于小于 n 的正整数命题为真, 那么

$$\begin{aligned} T(n) &= 2T(\lfloor \frac{n}{2} \rfloor) + n \leq 2c \lfloor \frac{n}{2} \rfloor \log(\lfloor \frac{n}{2} \rfloor) + n \\ &\leq 2c \frac{n}{2} (\log n - \log 2) + n = cn \log n - cn + n \\ &\leq cn \log n, \quad c=2 \end{aligned}$$

4. 递推方程的求解

迭代法

直接迭代: 插入排序最坏情况下时间分析

换元迭代: 二分归并排序最坏情况下时间分析 ①

差消迭代: 快速排序平均情况下的时间分析 ②

迭代模型: 递归树 ③

尝试法: 快速排序平均情况下的时间分析

主定理: 递归算法的分析

$$\begin{cases} W(n) = 2W(\frac{n}{2}) + n - 1, & n = 2^k \\ W(1) = 0 \end{cases}$$

$$W(n) = 2W(\frac{n}{2}) + n - 1$$

$$= 2[W(\frac{n}{2}) + \frac{n}{2} - 1] + n - 1$$

$$= 2^2 W(\frac{n}{2^2}) + 2n - (1+2)$$

$$= 2^3 [2W(\frac{n}{2^3}) + \frac{n}{4} - 1] + 2n - (1+2)$$

$$= 2^3 W(\frac{n}{2^3}) + 3n - (1+2+2^2)$$

$$= \dots$$

$$= 2^k W(1) + kn - (1+2+2^2+\dots+2^{k-1})$$

$$= kn - \frac{1-2^k}{1-2} = kn + 1 - 2^k = n \log n - n + 1$$

$$\begin{cases} T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + O(\log n), & n \geq 2 \\ T(1) = 0 \end{cases}$$

$$nT(n) = 2 \sum_{i=1}^{n-1} T(i) + cn^2$$

$$(n-1)T(n-1) = 2 \sum_{i=1}^{n-2} T(i) + C(n-1)^2$$

$$\Rightarrow nT(n) - (n-1)T(n-1) = 2T(n-1) + C(2n-1)$$

$$\Rightarrow nT(n) = (n+1)T(n-1) + C(2n-1)$$

$$\Rightarrow \frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{C_1}{n+1}$$

$$= \frac{T(n-1)}{n-1} + C_1 (\frac{1}{n+1} + \frac{1}{n})$$

$$= \dots$$

$$= \frac{T(1)}{2} + C_1 (\frac{1}{n+1} + \frac{1}{n} + \dots + \frac{1}{3})$$

$$= O(\log n)$$

$$\Rightarrow T(n) = O(n \log n)$$

主定理: $a \geq 1, b > 1$ 为常数, $T(n) \in \mathbb{N}^+$,

$$T(n) = aT(\frac{n}{b}) + f(n), \text{ 则有}$$

$$(1) f(n) = O(n^{\log_b a - \epsilon}), \epsilon > 0 \Rightarrow T(n) = \Theta(n^{\log_b a})$$

$$(2) f(n) = \Theta(n^{\log_b a}) \Rightarrow T(n) = \Theta(n^{\log_b a} \log n)$$

$$(3) f(n) = \Omega(n^{\log_b a + \epsilon}), \epsilon > 0$$

$$\exists 0 < c < 1, \text{ s.t. } af(\frac{n}{b}) \leq cf(n), \forall n \geq n_0$$

$$\Rightarrow T(n) = \Theta(f(n))$$

证: 不妨设 $n = b^k$

$$T(n) = aT(\frac{n}{b}) + f(n)$$

$$= a[aT(\frac{n}{b^2}) + f(\frac{n}{b})] + f(n)$$

$$= \dots$$

$$= a^k T(1) + f(n) + af(\frac{n}{b}) + \dots + a^{k-1} f(\frac{n}{b^{k-1}})$$

$$= a^k T(1) + \sum_{j=0}^{k-1} a^j f(\frac{n}{b^j})$$

$$= c_1 \cdot a^{\log_b n} + \sim$$

$$= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f(\frac{n}{b^j}) \quad (T(1) = c_1)$$

$$(1) f(n) = O(n^{\log_b a - \epsilon})$$

$$\Rightarrow T(n) = c_1 n^{\log_b a} + O(\sum_{j=0}^{k-1} a^j (\frac{n}{b^j})^{\log_b a - \epsilon})$$

$$= c_1 n^{\log_b a} + O(\sum_{j=0}^{k-1} a^j \frac{b^{\epsilon j}}{a^j} n^{\log_b a - \epsilon})$$

$$= c_1 n^{\log_b a} + O(n^{\log_b a - \epsilon} \sum_{j=0}^{k-1} b^{\epsilon j})$$

$$= c_1 n^{\log_b a} + O(n^{\log_b a - \epsilon} \frac{1 - b^{\epsilon k}}{1 - b^{\epsilon}})$$

$$= c_1 n^{\log_b a} + O(n^{\log_b a - \epsilon} \frac{b^{\epsilon \log_b n} - 1}{b^{\epsilon} - 1})$$

$$= c_1 n^{\log_b a} + O(n^{\log_b a - \epsilon} \cdot n^{\epsilon})$$

$$= \Theta(n^{\log_b a})$$

$$(2) f(n) = \Theta(n^{\log_b a})$$

$$\Rightarrow T(n) = c_1 n^{\log_b a} + \Theta(\sum_{j=0}^{k-1} a^j (\frac{n}{b^j})^{\log_b a})$$

$$= c_1 n^{\log_b a} + \Theta(\sum_{j=0}^{k-1} n^{\log_b a})$$

$$= c_1 n^{\log_b a} + \Theta(n^{\log_b a} \cdot (\log_b n - 1))$$

$$= \Theta(n^{\log_b a} \log n)$$

$$(3) f(n) = \Omega(n^{\log_b a + \epsilon}) \wedge \exists 0 < c < 1, af(\frac{n}{b}) \leq cf(n), n \geq n_0$$

$$T(n) = c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f(\frac{n}{b^j})$$

$$af(\frac{n}{b}) \leq cf(n)$$

$$\Rightarrow f(\frac{n}{b}) \leq \frac{c}{a} f(n)$$

$$\Rightarrow f(\frac{n}{b^j}) \leq \frac{c}{a} f(\frac{n}{b^{j-1}}) \leq \frac{c^j}{a^j} f(n)$$

...

$$\Rightarrow f(\frac{n}{b^j}) \leq \frac{c^j}{a^j} f(n)$$

$$\Rightarrow T(n) = c_1 n^{\log_b a} + \sum_{j=0}^{k-1} c^j f(n)$$

$$= c_1 n^{\log_b a} + f(n) \frac{1 - c^k}{1 - c}$$

$$= c_1 n^{\log_b a} + \Theta(f(n))$$

$$= \Theta(f(n))$$

例 求解递推方程 $T(n) = 3T(n/4) + n \log n$

上述递推方程中的 $a=3, b=4, f(n)=n \log n$, 那么

$$n \log n = \Omega(n^{\log_4 3 + \epsilon}) = \Omega(n^{0.793 + \epsilon}) \quad \epsilon \approx 0.2$$

此外, 要使 $af(n/b) \leq cf(n)$ 成立, 代入 $f(n)=n \log n$, 得到

$$\frac{3n}{4} \log \frac{n}{4} \leq cn \log n$$

显然只要 $c \geq 3/4$, 上述不等式就可以对充分大的 n 成立.

相当于主定理的第三种情况. 因此有

$$T(n) = \Theta(f(n)) = \Theta(n \log n)$$

不能使用主定理的例子

$$T(n) = 2T(n/2) + n \log n$$

$$T(1) = 1$$

□ 不能使用主定理

$$a=2, b=2, n^{\log_b a} = n, f(n) = n \log n$$

不存在 $c < 1$ 使得 $2(n/2) \log(n/2) \leq c n \log n$

不存在 $\epsilon > 0$ 使得 $f(n) = n \log n = \Omega(n^{1+\epsilon})$

□ 只能使用递归树, 结果等于

$$T(n) = n \log^2 n$$

课本习题

1.1 设 A 是 n 个不等的数的数组, n>2. 以比较作为基本运算, 试给出一个 O(1) 时间的算法找出 A 中一个既不是最大也不是最小的数. 写出算法的伪码, 说明该算法最坏情况下执行的比较次数.

```
1.1 A[0], A[1], A[2]
排序 -> B[0, 1, 2]
B[1]
W(n) = O(1)
if A[0] < A[1]:
    if A[1] < A[2]:
        return A[1]
    else if A[0] < A[2]:
        return A[2]
else
    return A[0]
```

1.2 考虑下述选择排序算法:

算法 ModSelectSort

输入: n 个不等的整数的数组 A[1..n]

输出: 按递增次序排序的 A

```
1. for i ← 1 to n-1 do
2.   for j ← i+1 to n do
3.     if A[j] < A[i] then A[i] ↔ A[j]
```

问:

(1) 最坏情况下该算法做多少次比较运算?

(2) 最坏情况下该算法做多少次交换运算? 这种情况在什么输入条件下发生?

1.2 解: 最坏情况为 A 递减排序

(1) 比较运算次数 = (n-1) + (n-2) + ... + 2 + 1

$$= \frac{[1+(n-1)](n-1)}{2}$$
$$= \frac{n(n-1)}{2}$$

(2) A 中 n 个数不等且递减排序为最坏情况, 交换次数也为 $\frac{n(n-1)}{2}$

1.3 给定正整数的数组 A[1..n], 测试 A 的每个元素 A[i] 的奇偶性. 如果 A[i] 是奇数, 则将它 2 倍后输出; 否则直接输出 A[i].

(1) 以乘法作为基本运算, 使用大 O 记号, 还是使用大 Θ 记号, 哪个记号能够正确表达这个算法对于规模为 n 的输入所做的基本运算次数? 为什么?

(2) 如果以元素的测试作为基本运算, 重复问题 (1).

1.3 (1) O.

我们只能估计出最坏情况下的基本运算次数,

(2) Θ.

Θ(n)

1.4 计算下述算法所执行的加法次数.

算法

输入: n = 2^t, t 为正整数

输出: k

```
1. k ← 0
2. while n ≥ 1 do
3.   for j ← 1 to n do
4.     k ← k + 1
5.   n ← n/2
6. return k
```

1.4 $2^t + 2^{t-1} + \dots + 2 + 1$

$$= \frac{1-2^{t+1}}{1-2} = 2^{t+1} - 1 = 2n - 1$$

1.5 计数算法 C 所执行的加法次数.

算法 C

输入: n 为正整数

输出: k

```
1. k ← 0
2. for i ← 1 to n do
3.   m ← ⌊n/i⌋
4.   for j ← 1 to m do
5.     k ← k + 1
6. return k
```

$\sum_{i=1}^n (\frac{n}{i} - 1) < \sum_{i=1}^n \lfloor \frac{n}{i} \rfloor \leq \sum_{i=1}^n \frac{n}{i}$

$n-1 \leq \lfloor n \rfloor \leq n$

1.5 $\lfloor \frac{n}{1} \rfloor + \lfloor \frac{n}{2} \rfloor + \dots + \lfloor \frac{n}{n} \rfloor$

$\approx n \sum_{i=1}^n \frac{1}{i} = \theta(n \log n)$

1.6 阅读关于下述算法 A 的伪码, 说明该算法求解的是什么问题, 并计算该算法所做的乘法运算 (*) 和加法运算次数.

算法 A

输入: 数组 P[0..n], 实数 x

输出: y

```
1. y ← P[0]; power ← 1
2. for i ← 1 to n do
3.   power ← power * x
4.   y ← y + P[i] * power
5. return y
```

1.6 解: 设 $P = [a_0, a_1, \dots, a_n]$.

则该算法求解 $\sum_{i=0}^n a_i x^i$, 即求解以 P 中元素为系数

的多项式 $f(x) = \sum_{i=0}^n a_i x^i$

乘法运算次数: 2n

加法运算次数: n

1.7 下述 Find-Second-Min 算法是找第二小算法. 输入是 n 个不等的数构成的数组 S, 输出是第二小的数 SecondMin.

(1) 在最坏情况下, 该算法做多少次比较?

(2) 若所有输入是等概率分布的, 平均情况下该算法做多少次比较?

算法 Find-Second-Min(S, n)

```
1. if S[1] < S[2]
2. then min ← S[1]; SecondMin ← S[2]
3. else min ← S[2]; SecondMin ← S[1]
4. for i ← 3 to n do
5.   if S[i] < SecondMin
6.   then if S[i] < min
7.     then SecondMin ← min; min ← S[i]
8.   else SecondMin ← S[i]
```

1.7 解: (1) 比较次数: $1 + 2 \times (n-3+1) = 1 + 2(n-2) = 2n-3$

(2) 考虑位置 i

若 S[i] 为 S[1..i] 中较小的 2 个数之一

则在 i 处需比较 2 次; 否则只需比较 1 次.

由于所有输入等概率分布, 故平均情况下比较次数为

$$1 + \sum_{i=3}^n (2 \times \frac{2}{i} + 1 \times \frac{i-2}{i})$$
$$= 1 + \sum_{i=3}^n (\frac{4}{i} + 1 - \frac{2}{i})$$
$$= 1 + (n-3+1) + 2 \sum_{i=3}^n \frac{1}{i}$$
$$= 1 + n - 2 + 2(I_n n + O(1))$$
$$= n - 1 + 2 I_n n + O(1)$$
$$= n + 2 I_n n + O(1)$$

(-4)

1.8 已知 L 是含有 n 个元素并且排好序的数组, x 在 L 中. 如果 x 出现在 L 中第 i 个 ($i = 2, 3, \dots, n$) 位置的概率是在前一个位置概率的一半, 当 n 充分大时, 估计下述查找算法平均情况下的时间复杂度 $A(n)$, 只需给出近似值.

算法 顺序查找

1. $j \leftarrow 1$
2. while $j \leq n$ and $x \neq L[j]$ do
3. $j \leftarrow j+1$
4. if $x < L[j]$ or $j > n$
5. then $j \leftarrow 0$

1.8 设在第 i 个位置的概率为 p_i ($i = 1, 2, \dots, n$)

$$A(n) = \sum_{i=1}^n i \cdot p_i$$

1.7, 1.8
理解平均复杂度

由题知:

$$p_{i+1} = \frac{1}{2} p_i \quad (i = 1, 2, \dots, n-1)$$

$$\Rightarrow \frac{p_i (1 - p_i^n)}{1 - \frac{1}{2}} = 1$$

$$\Rightarrow p_i (1 - p_i^n) = \frac{1}{2}$$

故当 $n \rightarrow +\infty$ 时, $p_i = \frac{1}{2}$ ($p_i < 1$)

$$\Rightarrow p_i = \left(\frac{1}{2}\right)^i$$

$$\Rightarrow A(n) = \sum_{i=1}^n \frac{i}{2^i} \quad (\text{当 } n \text{ 充分大时})$$

$$\Delta A(n) = \sum_{i=1}^n \frac{i}{2^{i-1}}$$

$$= \sum_{i=0}^{n-1} \frac{i+1}{2^i}$$

$$= \sum_{i=0}^{n-1} \frac{i}{2^i} + \sum_{i=0}^{n-1} \frac{1}{2^i}$$

$$\Rightarrow A(n) = \Delta A(n) - A(n)$$

$$= -\frac{n}{2^n} + \frac{1(1 - \frac{1}{2^n})}{1 - \frac{1}{2}}$$

$$= -\frac{1+n}{2^n} \Rightarrow$$

$$\Rightarrow \lim_{n \rightarrow +\infty} A(n) = 2$$

1.9 设 A 为 n 个不等的数的数组. 给定 x , 若 x 在 A 中, 输出 x 的下标 k ; 若 x 不在 A 中, 输出 0. BinarySearch 和 LinearSearch 分别表示二分和顺序搜索, 设计下述算法:

算法 Search(A, x)

1. if n 为奇数 then $k \leftarrow \text{BinarySearch}(A, x)$
2. else $k \leftarrow \text{LinearSearch}(A, x)$

以比较作基本运算, 用大 O 记号表示算法在最坏情况下的时间复杂度 $W(n)$; 能否使用大 Θ 记号表示 $W(n)$? 为什么?

$$1.9 \quad W(n) = O(n)$$

不能, $W(n) = \Theta(\lg n)$, n 奇. 此时 $W(n) \neq \Omega(n)$

1.10 考虑下述素数测试算法:

算法 PrimalityTest(n)

输入: n , n 为大于 2 的奇整数

输出: true 或者 false

1. $s \leftarrow \lfloor \sqrt{n} \rfloor$
2. for $j \leftarrow 2$ to s
3. if j 整除 n
4. then return false
5. return true

(1) 假设计算 $\lfloor \sqrt{n} \rfloor$ 可以在 $O(1)$ 时间完成, 估计该算法在最坏情况下的时间复杂度.

(2) 能否使用 Θ 符号表示这个算法在最坏情况下的时间复杂度? 为什么?

$$1.10 \quad (1) \quad O(n^{\frac{1}{2}})$$

(2) 不能, n 为奇数 $O(1)$, $O(1) \neq \Omega(n^{\frac{1}{2}})$

1.11 证明定理 1.3: 假设 f 和 g 是定义域为自然数集合的函数, 若对某个其他函数 h , 有 $f = O(h)$ 和 $g = O(h)$ 成立, 那么 $f + g = O(h)$.

$$1.11 \quad \exists c_1, n_0 > 0, \text{ s.t. ,}$$

$$f(n) \leq c_1 h(n), \quad n > n_0$$

$$\exists c_2, n'_0 > 0, \text{ s.t. ,}$$

$$g(n) \leq c_2 h(n), \quad n > n'_0$$

$$\Rightarrow f(n) + g(n) \leq (c_1 + c_2) h(n), \quad n > \max\{n_0, n'_0\}$$

$$\text{取 } C = c_1 + c_2, \quad m = \max\{n_0, n'_0\}$$

1.12 证明定理 1.4: 对每个 $b > 1$ 和每个 $a > 0$, 有 $\log n = o(n^a)$.

$$1.12 \quad \lim_{n \rightarrow \infty} \frac{\log n}{n^a} = \lim_{n \rightarrow \infty} \frac{\frac{1}{n}}{a n^{a-1}} = \lim_{n \rightarrow \infty} \frac{1}{a n^a} \cdot \frac{1}{\ln b} = 0$$

洛

$$\log n, n^a, r^n, a > 0, r > 1$$

1.13 证明定理 1.5: 对每个 $r > 1$ 和每个 $d > 0$, 有 $n^d = o(r^n)$.

$$1.13 \quad \lim_{n \rightarrow \infty} \frac{n^d}{r^n} = \lim_{n \rightarrow \infty} \frac{d n^{d-1}}{\ln r \cdot r^n} = \dots = \lim_{n \rightarrow \infty} \frac{d(d-1) \dots 2n}{(\ln r)^{d-1} r^n} \\ = \lim_{n \rightarrow \infty} \frac{\frac{d!}{i!} i}{(\ln r)^d r^n} = 0$$

$$d \leq 1, \lim_{n \rightarrow \infty} \frac{n^d}{r^n} = \lim_{n \rightarrow \infty} \frac{d \cdot n^{d-1}}{\ln r \cdot r^n} = 0$$

$$d > 1, \lim_{n \rightarrow \infty} \frac{n^d}{r^n} = \dots = \lim_{n \rightarrow \infty} \frac{n^{d_0}}{r^n} \quad (c > 0, d_0 < 1) \\ = 0$$

1.14 设 x 为实数, n, a, b 为整数, 证明下述性质:

(1) $x-1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x+1$

(2) $\lfloor x+n \rfloor = \lfloor x \rfloor + n, \lceil x+n \rceil = \lceil x \rceil + n$

$$(3) \left\lfloor \frac{n}{2} \right\rfloor + \left\lceil \frac{n}{2} \right\rceil = n$$

$$(4) \left\lfloor \frac{\frac{n}{a}}{b} \right\rfloor = \left\lfloor \frac{n}{ab} \right\rfloor, \left\lceil \frac{\frac{n}{a}}{b} \right\rceil = \left\lceil \frac{n}{ab} \right\rceil$$

1.14 设 $x = M + m$, 其中 M 为 x 的整数部分, m 为 x 的小数部分

$$M \in \mathbb{Z}, \quad 0 \leq m < 1$$

(1) ① $x \in \mathbb{Z}$, 显然, 结论成立

② x 为小数, 即 $m \neq 0$,

$$x-1 = M-1 + m$$

$$\lfloor x \rfloor = M$$

$$\lceil x \rceil = M+1$$

$$x+1 = M+1 + m$$

$$\text{有 } x-1 < \lfloor x \rfloor \leq x < \lceil x \rceil < x+1$$

■

(2) ① $m = 0, x = M$

$$\lfloor x+n \rfloor = M+n = x+n = \lfloor x \rfloor + n$$

$$\lceil x+n \rceil = M+n = x+n = \lceil x \rceil + n$$

② $m \neq 0$

$$\lfloor x+n \rfloor = \lfloor M+m+n \rfloor = M+n = \lfloor x \rfloor + n$$

$$\lceil x+n \rceil = \lceil M+m+n \rceil = M+1+n = \lceil x \rceil + n$$

■

(3) ① $n = 2k, k \in \mathbb{Z}$

$$\frac{n}{2} = k, \quad \left\lfloor \frac{n}{2} \right\rfloor + \left\lceil \frac{n}{2} \right\rceil = 2k = n$$

② $n = 2k+1, k \in \mathbb{Z}$

$$\frac{n}{2} = k + \frac{1}{2}$$

$$\left\lfloor \frac{n}{2} \right\rfloor + \left\lceil \frac{n}{2} \right\rceil = k+1 + k = 2k+1$$

■

1.4) 设 $\frac{n}{a} = M_1 + m_1, M_1 \in \mathbb{Z}, 0 \leq m_1 < 1$

① $m_1 = 0, \frac{n}{a} = M_1$

$\left\lceil \frac{n}{a} \right\rceil = \left\lfloor \frac{n}{a} \right\rfloor = M_1$

$\Rightarrow \left\lceil \frac{\left\lfloor \frac{n}{b} \right\rfloor}{b} \right\rceil = \left\lfloor \frac{n}{ab} \right\rfloor$

$\left\lfloor \frac{\left\lceil \frac{n}{a} \right\rceil}{b} \right\rfloor = \left\lfloor \frac{n}{ab} \right\rfloor$

② $m_1 \neq 0, \frac{n}{a} = M_1 + m_1$

$\left\lceil \frac{n}{a} \right\rceil = M_1 + 1$

$\left\lfloor \frac{n}{a} \right\rfloor = M_1$

$\Rightarrow \left\lceil \frac{\left\lfloor \frac{n}{b} \right\rfloor}{b} \right\rceil = \left\lfloor \frac{M_1 + 1}{b} \right\rfloor = \left\lfloor \frac{\frac{n}{a} - m_1 + 1}{b} \right\rfloor$
 $= \left\lfloor \frac{n}{ab} + \frac{1 - m_1}{b} \right\rfloor$

带余除法

设 $n = pab + r, 0 \leq r < ab, p \in \mathbb{Z}$

① $r = 0 \checkmark$

② $r \neq 0, \left\lceil \frac{n}{a} \right\rceil = \left\lfloor pb + \frac{r}{a} \right\rfloor = pb + \left\lfloor \frac{r}{a} \right\rfloor$

$\left\lfloor \frac{\left\lceil \frac{n}{a} \right\rceil}{b} \right\rfloor = \left\lfloor p + \frac{\left\lfloor \frac{r}{a} \right\rfloor}{b} \right\rfloor = p + \left\lfloor \frac{\left\lfloor \frac{r}{a} \right\rfloor}{b} \right\rfloor = p$

$\left\lfloor \frac{n}{ab} \right\rfloor = p + \left\lfloor \frac{r}{ab} \right\rfloor = p$

1.15 考虑下面每对函数 $f(n)$ 和 $g(n)$, 如果它们的阶相等则使用 Θ 记号, 否则使用 O 记号表示它们的关系。

(1) $f(n) = (n^2 - n)/2, g(n) = 6n$

(2) $f(n) = n + 2\sqrt{n}, g(n) = n^2$

(3) $f(n) = n + n \log n, g(n) = n\sqrt{n}$

(4) $f(n) = 2 \log^2 n, g(n) = \log n + 1$

(5) $f(n) = \log(n!), g(n) = n^{1.05}$

1.15 解: (1) $g(n) = O(f(n))$

(2) $f(n) = O(g(n))$

(3) $f(n) = O(g(n))$

(4) $g(n) = O(f(n))$

(5) $f(n) = O(g(n))$

1.16 在表 1.1 中填入 true 或 false.

表 1.1 函数 f 与 g

函数序号	$f(n)$	$g(n)$	$f(n) = O(g(n))$	$f(n) = \Omega(g(n))$	$f(n) = \Theta(g(n))$
1	$2n^3 + 3n$	$100n^2 + 2n + 100$	☒	☒	☒
2	$50n + \log n$	$10n + \log \log n$	☒	☒	☒
3	$50n \log n$	$10n \log \log n$	☒	☒	☒
4	$\log n$	$\log^2 n$	☒	☒	☒
5	$n!$	5^n	☒	☒	☒

1.17 对于下面每个函数 $f(n)$, 用 Θ 符号表示成 $f(n) = \Theta(g(n))$ 的形式, 其中 $g(n)$ 要尽可能简洁. 比如 $f(n) = n^2 + 2n + 3$ 可以写为 $f(n) = \Theta(n^2)$. 然后, 按照阶递增的顺序将这些函数进行排列.

$(n-2)!, 5 \log(n+100)^{10}, 2^{2n}, 0.001n^4 + 3n^3 + 1,$

$(\ln n)^2, \sqrt[3]{n} + \log n, 3^n, \log(n!), \log(n^{n+1}), 1 + \frac{1}{2} + \dots + \frac{1}{n}$

1.17 ① $(n-2)! = \Theta(n!)$

$5 \log(n+100)^{10} = \Theta(\log n)$

$2^{2n} = \Theta(4^n)$

$0.001n^4 + 3n^3 + 1 = \Theta(n^4)$

$\ln n = \Theta(\log n)$

$\sqrt[3]{n} + \log n = \Theta(\sqrt[3]{n})$

$3^n = \Theta(3^n)$

$\log(n!) = \Theta(n \log n)$

$\log n^{n+1} = \Theta(n \log n)$

$1 + \frac{1}{2} + \dots + \frac{1}{n} = \Theta(\log n)$

② $1 + \frac{1}{2} + \dots + \frac{1}{n}, 5 \log(n+100)^{10} = \Theta(\log n)$

$\ln n = \Theta(\log n)$

$\log(n!), \log n^{n+1} = \Theta(n \log n)$

$\sqrt[3]{n} + \log n, n^4$

$3^n, 2^{2n} = \Theta(4^n)$

$(n-2)! = \Theta(3^n), \Theta(4^n)$

1.18 对以下函数, 按照它们的阶从高到低排列; 如果 $f(n)$ 与 $g(n)$ 的阶相等, 表示为 $f(n) = \Theta(g(n))$.

$2^{\sqrt{2} \log n}, n \log n, \sum_{k=1}^n \frac{1}{k}, n 2^n, (\log n)^{\log n}, 2^{2n}, 2^{\log \sqrt{n}}$

$n^3, \log(n!), \log n, \log \log n, n^{\log \log n}, n!, n, \log 10^n$

1.18 解: $n!, 2^{2n}, n 2^n, n^{\log \log n} = \Theta(\log n^{\log n})$

$n^3, \log(n!) = \Theta(n \log n), \log 10^n = \Theta(n), 2^{\log \sqrt{n}}, 2^{\sqrt{2} \log n}$

$\sum_{k=1}^n \frac{1}{k} = \Theta(\log n), \log \log n$

1.19 求解以下递推方程:

(1) $\begin{cases} T(n) = T(n-1) + n^2 \\ T(1) = 1 \end{cases}$

(2) $\begin{cases} T(n) = 9T(n/3) + n \\ T(1) = 1 \end{cases}$

(3) $\begin{cases} T(n) = T(n/2) + T(n/4) + cn, \quad c \text{ 为常数} \\ T(1) = 1 \end{cases}$

(4) $\begin{cases} T(n) = T(n-1) + \log 3^n \\ T(1) = 1 \end{cases}$

(5) $\begin{cases} T(n) = 5T(n/2) + (n \log n)^2 \\ T(1) = 1 \end{cases}$

(6) $\begin{cases} T(n) = 2T(n/2) + n^2 \log n \\ T(1) = 1 \end{cases}$

(7) $\begin{cases} T(n) = T(n-1) + 1/n \\ T(1) = 1 \end{cases}$

(8) $T(n) = T(n-1) + \log n$, 估计 $T(n)$ 的阶.

1.19 (1) $T(n) = T(n-1) + n^2$

$= T(n-2) + n^2 + (n-1)^2$

$= T(n-3) + n^2 + (n-1)^2 + (n-2)^2$

$= \dots$

$= T(1) + n^2 + (n-1)^2 + \dots + 2^2$

$= \sum_{i=1}^n i^2$

$= \frac{n(n+1)(2n+1)}{6}$

$$(2) T(n) = 9T(\frac{n}{3}) + n$$

$$= 9[9T(\frac{n}{3^2}) + \frac{n}{3}] + n$$

$$= 9^2 T(\frac{n}{3^2}) + 3n + n$$

$$= 9^2 [9T(\frac{n}{3^3}) + \frac{n}{3^2}] + 3n + n$$

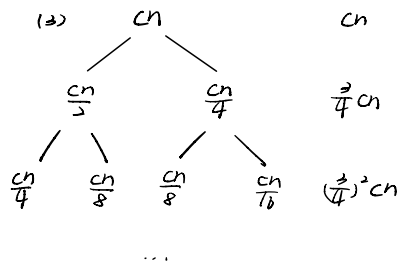
$$= 9^3 T(\frac{n}{3^3}) + 9n + 3n + n$$

$$= \dots$$

$$(设 n = 3^k) = 9^k T(1) + (3^{k-1} + 3^{k-2} + \dots + 1) n$$

$$= 9^k + \frac{1-3^k}{1-3} n$$

$$= n^2 + \frac{n-1}{2} n = \frac{3}{2} n^2 - \frac{1}{2} n = \theta(n^2)$$



$$T(n) = [1 + \frac{7}{4} + (\frac{7}{4})^2 + \dots] cn = \theta(n)$$

$$(4) T(n) = T(n-1) + \log 3^n$$

$$= T(n-2) + \log 3^{n-1} + \log 3^n$$

$$= \dots$$

$$= T(1) + \log 3^2 + \log 3^3 + \dots + \log 3^n$$

$$= 1 + \log 3^{2+3+\dots+n}$$

$$= 1 + \log 3^{\frac{(n+2)(n-1)}{2}}$$

$$= 1 + \frac{n^2+n-2}{2} \log 3$$

$$(5) a=5, b=2$$

$$f(n) = (n \log n)^2 = O(n^{\log 5 - \epsilon})$$

$$由主定理: T(n) = \theta(n^{\log 5})$$

$$(6) T(n) = 2T(\frac{n}{2}) + n^2 \log n$$

$$a=2, b=2, n^{\log a} = n$$

$$f(n) = n^2 \log n = \Omega(n^{\log a + \epsilon}), \epsilon > 0,$$

$$取 c = \frac{1}{2} < 1$$

$$2f(\frac{n}{2}) = 2(\frac{n^2}{4} \log \frac{n}{2}) = \frac{n^2}{2} \log \frac{n}{2} \leq cn^2 \log n, n \rightarrow +\infty$$

$$故 T(n) = \theta(f(n)) = \theta(n^2 \log n)$$

$$(7) T(n) = T(n-1) + \frac{1}{n}$$

$$= T(n-2) + \frac{1}{n-1} + \frac{1}{n}$$

$$= T(n-3) + \frac{1}{n-2} + \frac{1}{n-1} + \frac{1}{n}$$

$$= \dots$$

$$= T(1) + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$$

$$= \sum_{i=1}^n \frac{1}{i}$$

$$= \theta(\log n)$$

$$(8) T(n) = T(n-1) + \log n$$

$$= T(n-2) + \log(n-1) + \log n$$

$$= T(n-3) + \log(n-2) + \log(n-1) + \log n$$

$$= \dots$$

$$= T(1) + \log \frac{n!}{1}$$

$$= T(1) + \log(n!)$$

$$= \theta(n \log n)$$

1.20 设递推方程 $T(n) = 7T(n/2) + n^2$ 给出了算法 A 在最坏情况下的时间复杂度函数, 算法 B 在最坏情况下的时间复杂度函数 $W(n)$ 满足递推方程 $W(n) = aW(n/4) + n^2$, 试确定最大的正整数 a 使得 $W(n)$ 的阶低于 $T(n)$ 的阶.

$$1.20 T(n) = 7T(\frac{n}{2}) + n^2$$

$$a=7, b=2$$

$$n^2 = O(n^{\log 7 - \epsilon})$$

$$\Rightarrow T(n) = \theta(n^{\log 7})$$

$$W(n) = aW(\frac{n}{4}) + n^2$$

$$\textcircled{1} n^2 = O(n^{\log 4a - \epsilon})$$

$$\Rightarrow a > 16$$

$$此时 W(n) = \theta(n^{\log 4a})$$

$$\Rightarrow \sqrt{a} < 7 \Rightarrow a < 49 \quad \log 4a = \log 4 + \log a < \log 7$$

$$故 16 < a < 49$$

$$\textcircled{2} n^2 = \theta(n^{\log 4a}) \Rightarrow a = 16$$

$$此时 W(n) = \theta(n^2)$$

$$故 a = 16$$

$$\textcircled{3} n^2 = \Omega(n^{\log 4a + \epsilon}) \quad (c < 1)$$

$$\exists c, a(\frac{n}{4})^2 \leq cn^2, n \rightarrow \infty$$

$$\Rightarrow a < 16 \quad (此时取 c=1 即可)$$

综上 $0 < a < 49$ 时均满足题意

故最大的正整数 $a = 48$.

1.21 设原问题的规模是 n , 从下述三个算法中选择一个最坏情况下时间复杂度最低的算法, 简要说明理由.

算法 A: 将原问题划分规模减半的 5 个子问题, 递归求解每个子问题, 然后在线性时间将子问题的解合并得到原问题的解.

算法 B: 先递归求解 2 个规模为 $n-1$ 的子问题, 然后在常量时间内将子问题的解合并.

算法 C: 将原问题划分规模为 $n/3$ 的 9 个子问题, 递归求解每个子问题, 然后在 $O(n^3)$ 时间将子问题的解合并得到原问题的解.

1.21 解: 设问题规模为 n

① 算法 A:

$$T(n) = 5T\left(\frac{n}{5}\right) + cn \quad (c > 0)$$

$$a=5, b=5, f(n) = O(n^{\log_5 5 - \epsilon}), \epsilon > 0 \text{ 存在}$$

由主定理:

$$T(n) = \theta(n^{\log_5 5})$$

② 算法 B:

$$T(n) = 2T(n-1) + c \quad (c > 0)$$

$$= 2[2T(n-2) + c] + c$$

$$= 2^2 T(n-2) + 2c + c$$

$$= 2^2 [2T(n-3) + c] + 2c + c$$

$$= \dots$$

$$= 2^{n-1} T(1) + (2^{n-1} + \dots + 2 + 1)c$$

$$= 2^{n-1} c' + \frac{1-2^n}{1-2} c \quad (c' > 0)$$

$$= \theta(2^n)$$

③ 算法 C:

$$T(n) = 9T\left(\frac{n}{3}\right) + cn^3 \quad (c > 0)$$

$$a=9, b=3, n^{\log_3 9} = n^2$$

$$f(n) = \Omega(n^{\log_3 9 + \epsilon}), \text{ 取 } \epsilon = \frac{1}{3} > 0,$$

$$af\left(\frac{n}{b}\right) = 9f\left(\frac{n}{3}\right) = 9 \cdot \frac{n^3}{27} \cdot c = \frac{c}{3} n^3$$

$$\text{取 } c' = \frac{1}{3} \leq 1,$$

$$af\left(\frac{n}{b}\right) = c' f(n) = \frac{c}{3} n^3, \quad n \rightarrow \infty,$$

$$\text{故 } T(n) = \theta(n^3)$$

故应选择算法 A.

补充题

求解证明题1: 求解递推方程并证明结论

求 $T(n) = 2T(n/2) + n/\log n$ 的解, 并用代入归纳法证明。

1. 解: $T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log n}$

设 $n = 2^k$ *换元*

$$T(2^k) = 2T(2^{k-1}) + \frac{2^k}{k}$$

$$= 2\left[2T(2^{k-2}) + \frac{2^{k-1}}{k-1}\right] + \frac{2^k}{k}$$

$$= 2^2 T(2^{k-2}) + 2^k \left(\frac{1}{k-1} + \frac{1}{k}\right)$$

$$= \dots$$

$$= 2^{k-1} T(1) + 2^k \sum_{i=2}^k \frac{1}{i}$$

$$= c \cdot \frac{n}{2} + n(\ln k + O(1))$$

$$= \frac{c}{2} n + n \ln \log n + O(n)$$

$$= \theta(n \log \log n) \quad \text{此题可以用主定理吗?}$$

不用。

$$\text{故 } T(n) = \theta(n \log \log n)$$

△ 下用归纳法证明:

$$\text{设 } T(k) = \theta(k \log \log k) \text{ 对 } n \leq k \text{ 成立, } k \in \mathbb{Z}_+$$

则当 $n = k+1$ 时

$$T(k+1) = 2T\left(\frac{k+1}{2}\right) + \frac{k+1}{\log(k+1)}$$

$$= \theta((k+1) \log \log(k+1)) + \frac{k+1}{\log(k+1)}$$

$$= \theta((k+1) \log \log(k+1))$$

$$\text{故对 } \forall k \in \mathbb{Z}_+, T(k) = \theta(k \log \log k)$$

$$\text{故 } T(n) = \theta(n \log \log n) \quad \blacksquare$$

求解证明题2: 证明递推方程的解

证明 $T(n) = T(n/2) + 1$ 的解是 $O(\log n)$ (注: 不能直接使用主定理)。

2. 证: $T(n) = T\left(\frac{n}{2}\right) + 1$

$$= T\left(\frac{n}{4}\right) + 1 + 1$$

$$= T\left(\frac{n}{8}\right) + 1 + 1 + 1$$

$$= \dots$$

(设 $n = 2^k$) $= T(1) + k$

$$= T(1) + \log n = O(\log n) \quad \blacksquare$$